PROCESS WATCHDOG

OS Hackathon Project

Section- 10
Batch- 2

2520090001- Kamtam Priyanka
2520090123- Jalla Geya Reddy

Under the estimed guidance of
M. Raghupathi

Introduction

A practical bridge between OS theory and observable process behaviour.

01

Monitor

Observe whether an independent worker process is running.

02

Detect

Recognize an unexpected exit through periodic status checks.

03

Recover

Attempt a controlled restart under a configurable policy.

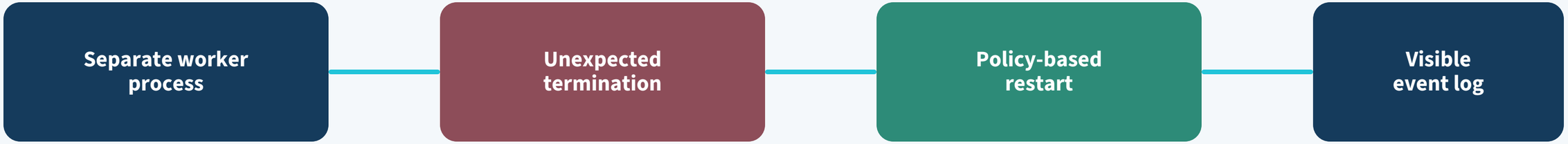
04

Learn

Connect processes, concurrency, synchronization and logs to practice.

Problem Statement

A lightweight supervisor is needed—simple enough to study, complete enough to demonstrate.



How can one app launch, watch, recover and explain another process—without hiding the operating-system mechanisms?

Six Project Objectives

01

Manage a worker process

02

Monitor state periodically

03

Detect abnormal termination

04

Restart within policy limits

05

Protect shared state with locks

06

Expose events in logs + dashboard

Proposed System

One dashboard. One watchdog. A traceable recovery loop.



MONITORING CONTROLS
Start / Stop monitoring

RESTART POLICY
Limit · delay · enable

CRASH SIMULATION
Terminate worker safely

EVENTS
Live + persistent logs

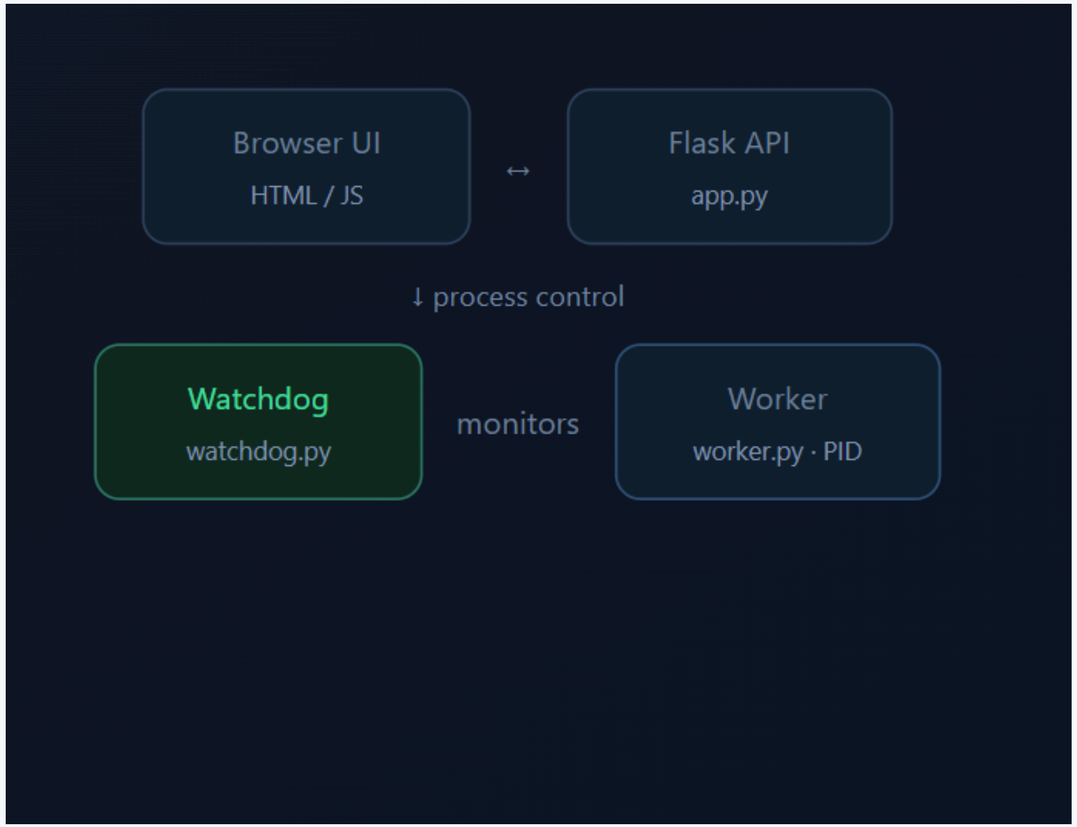
Technologies Used

Each choice has one clear role in the system.

Python + subprocess	Launch and inspect the worker process
Flask	Serve APIs and the web dashboard
HTML · CSS · JavaScript	Render status, controls and event updates
Ubuntu / WSL2	Provide the Linux execution environment
threading + locks	Run watchdog checks and synchronize state
File logging	Persist process and recovery events

System Architecture

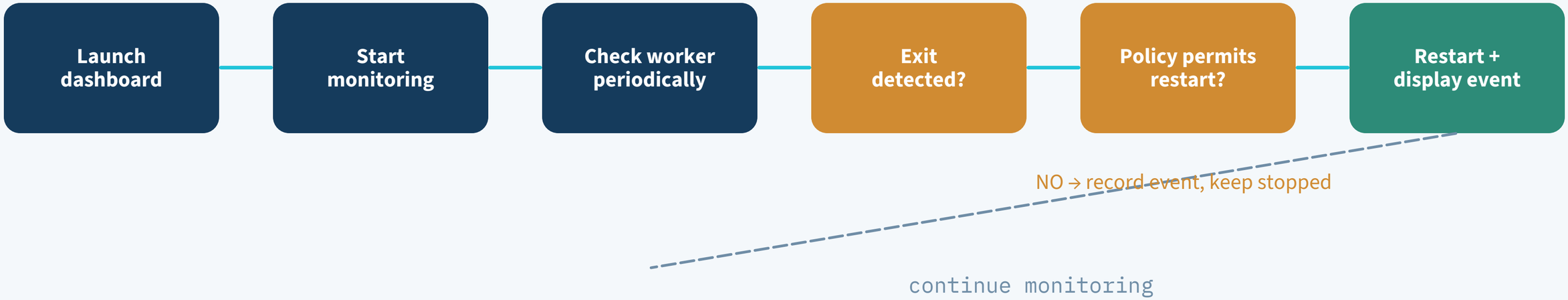
Replace the marked area with the exact user-provided architecture diagram. Do not redraw or approximate it.



Expected layers: Browser → Dashboard → Flask app → Watchdog thread → Worker process → Linux OS facilities → Persistent event log

System Workflow

Editable control flow from launch to visible recovery events.



Modules & Folder Structure

Suggested structure—rename entries to match the submitted repository.

```
project/
├── app.py
├── watchdog.py
├── worker.py
├── config.json
├── templates/index.html
├── static/style.css
├── static/app.js
└── logs/events.log
```

app.py · routes, APIs, application state

watchdog.py · polling, exit detection, restart policy

worker.py · independently launched demo workload

config.json · monitoring interval and restart settings

index.html · dashboard structure

style.css / app.js · interface and actions

events.log · persistent operational history

Implementation & Core Logic

The watchdog keeps recovery policy separate from process execution.

```
WATCHDOG LOOP

lock(state)
if worker is absent:
    launch via subprocess

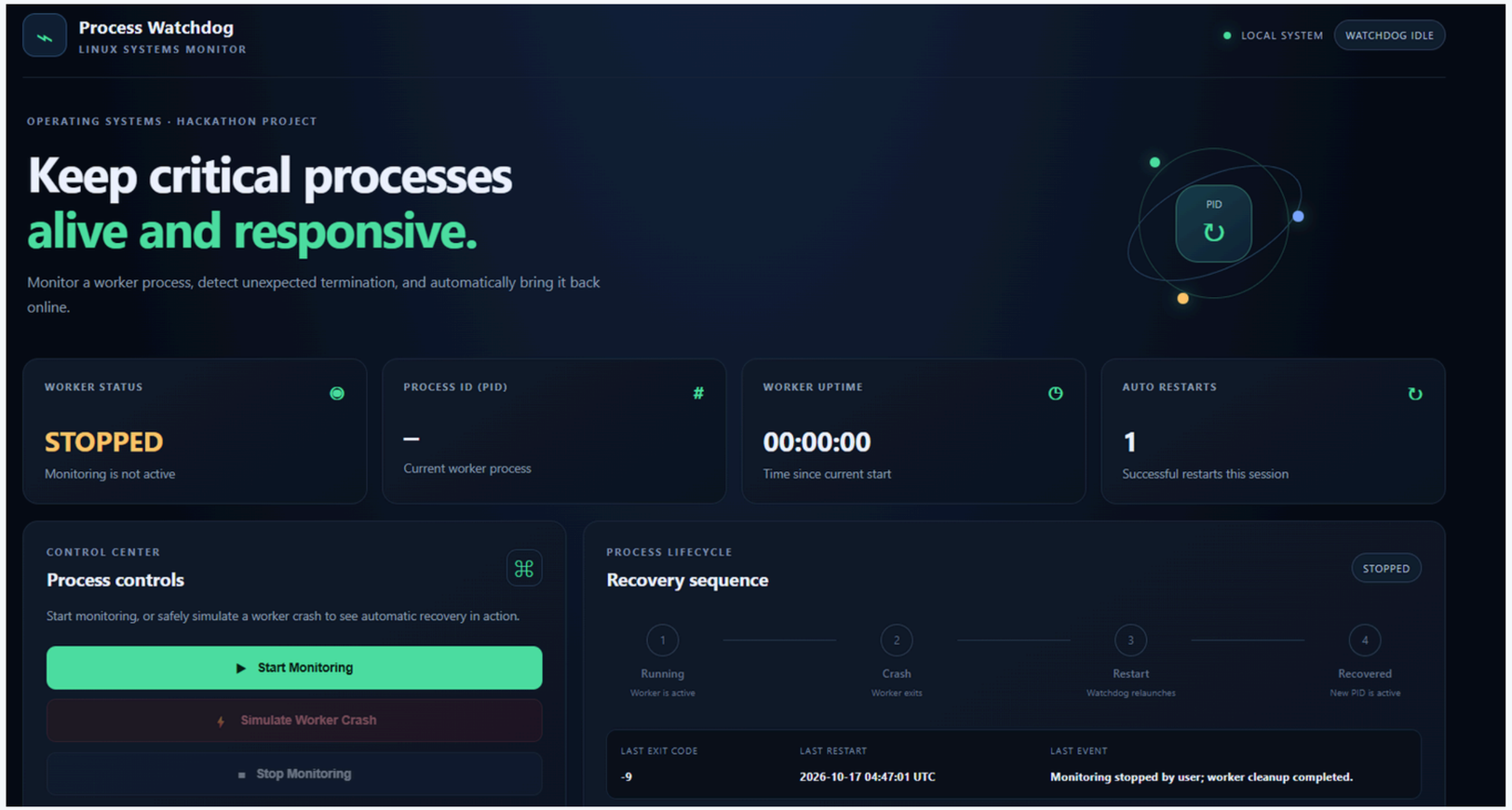
repeat every interval:
    code = worker.poll()
    if code is not None:
        log(exit code, timestamp)
        wait(restart_delay)
        if policy.allows(): restart()
    publish snapshot()
```

CORE MECHANISMS

01 subprocess launch
02 periodic liveness checks
03 exit-code detection
04 restart delay + limit
05 lock-protected shared state
06 file + dashboard event logging

Dashboard & User Interaction

The screenshot area is intentionally reserved for the final build—no fabricated UI capture.



VISIBLE INFORMATION

Status + PID

Uptime

Restart count

Recent logs

USER CONTROLS

Start / stop monitoring

Crash simulation

Restart policy

Polling interval

Testing & Demonstration

Test plan only: expected behaviour is listed; no measured results are claimed.

Action	Expected result
Launch dashboard	Page loads; worker state is visible
Start monitoring	Watchdog begins periodic checks
Simulate crash	Worker exits; termination event appears
Enable restart policy	Permitted restart creates a new PID
Reach restart limit	Worker remains stopped; reason is logged
Refresh browser	Current state and recent logs remain visible

OS Concepts & Course Outcomes

Directly implemented mechanisms are separated from concepts covered through explanation.

CO-1	Processes, states, termination	DIRECT
CO-2	Scheduling and lifecycle observation	DIRECT
CO-3	Threads, locks, shared state	DIRECT
CO-4	Deadlock / memory concepts	THEORY
CO-5	File-based persistent logging	DIRECT
CO-6	Security, protection, IPC concepts	THEORY

Results, Limitations & Future

A clear boundary between demonstrated behaviour and proposed extensions.

EXPECTED BEHAVIOUR

Detect worker exit
Attempt policy-based restart
Update state and logs
Show events on dashboard

CURRENT LIMITATIONS

Single-host prototype
Polling—not event-driven
Basic authentication absent
Limited process scope

FUTURE ONLY

Multiple workers
Notifications + metrics
Role-based access
Container deployment

From failure to visible recovery

The project turns core OS mechanisms into an observable, explainable system.



Thank You